

Computational Finance with Map-Reduce in Scala

Ron Coleman, Udaya Ghattamaneni, Mark Logan, and Alan Labouseur

Computer Science Department

Marist College

Poughkeepsie, NY, 12601

Abstract – This paper presents results of computational finance experiments using map-reduce in Scala. We observe superlinear speedup, super-efficiency, and evidence for a high degree of compute and I/O overlap in the median runtimes using “naïve,” memory-bound, fine-grain, and coarse-grain parallel algorithms on three different hardware platforms.

Keywords - Computational finance, map-reduce, Scala, actors, parallel programming, bond pricing, securities pricing

1 Introduction

Computational finance is a multidisciplinary field at the crossroads of mathematical finance and computer science. [29] The emphasis is on development and utilization of numerically intensive methods for pricing, risk analysis, forecasting, automated trading, and other applications. Map-reduce [3] is a framework generally to speed-up data analysis using distributed computing. While map-reduce has been applied to different problem domains, many of a data-intensive nature, almost no attention has been given to opportunities for computational finance as a mixture of floating point and data-intensive operations.

Scala [23] is a modern, high-level Java Virtual Machine (JVM) language that blends object-oriented and functional programming styles with actors, a shared-nothing model of concurrent computation inspired by physics theories. [13] Proponents have argued that Scala language features are suited to solving large-scale computing tasks on inexpensive, commodity multicore and multiprocessor platforms [11] in an expressive manner that avoids the concurrency hazards and runtime inefficiencies of shared, mutable state programs. Indeed, the function-oriented style of Scala would seem to lend itself precisely to coding mathematical expressions which characterize quantitative operations.

We endeavored to put Scala’s scalability claims to the test on a large quantitative problem, namely, finding the fair value of bond portfolios. We chose bonds because fixed income pricing theory is fairly transparent. [6, 14, 29] Furthermore, the price of a bond informs or is closely related to a number of different financial

instruments, including annuities, bond derivatives, and interest rate swaps, the most heavily traded financial derivative in the world. [14] Thus, we posit that the practical performance implications of bond portfolio analysis extend to scientific and technical computing.

In this paper, we study several parallel algorithms in relation to two serial algorithms to price bond portfolios using map-reduce in Scala. For three different multicore platforms, the data generally show superlinear speedup and super-efficiency [9] in the median runtime that is in line with the maximum performance expectations published by the original equipment manufacturer (OEM) [15]. The data show that the parallel “naïve” algorithm, which assigns one portfolio per map function object, has median performance little different from, if not better than, memory-bound, fine-grain, and coarse-grain parallel algorithms. In other words, the Scala actor model, which underlies map-reduce, appears to exploit hardware threads (i.e., “hyper-threads”) very efficiently. To our knowledge these findings have not been reported elsewhere.

The Scala source code we used in this study and detailed experimental results are available online [1,7].

2 Related work

The literature shows enduring interest in speeding up computational finance algorithms [2,22,26], most notably using specialized hardware and ad hoc low-level programming techniques [19,22], which do not, in our experience and opinion as programmers, promote software reusability or programmer productivity. The literature furthermore indicates map-reduce is a widely accepted approach to speeding up computation for various problem classes, although many of these efforts appear to be of a data-intensive nature. [3,8,16,27] Few, to our knowledge, have investigated computational finance applications. Zhang, et al [32] investigates map-reduce for option pricing using the backward stochastic differential equation (BSDE) method. However, the focus there is on the computational aspect of solving BSDE; we consider the end-to-end process, namely,

compute and I/O phases that characterize more practical settings. Woo and Xu [31] use map-reduce to perform market basket analysis with Amazon's EC2 service. This effort is geared toward data analysis, not computational finance; and cloud computing rather than parallel processing (i.e., speedup and efficiency) which is our primary focus. There is at least one cloud map-reduce framework for JVM languages [10], although this framework has have not been specifically designed for Scala. We instead use a pure-Scala map-reduce API from Haller and Sommers (H-S) [12]. Scala also has a library which is not actor-based in which it is also possible to employ map and reduce parallel functions directly on parallel-enabled collections (e.g., lists, arrays, etc.). [25] We do not investigate parallel collections here.

3 Methods

In this section we develop the methods of the paper. We discuss of bond pricing theory, portfolio valuation, the serial processing of portfolios in Scala, and the parallel processing of portfolios using map-reduce in Scala.

3.1 Bond pricing theory

A bond is a fixed income security. [6, 14, 29] For our purposes in this paper, we are considering only simple bonds, b_i , defined by the 5-tuple:

$$b_i = [i, C, n, T, M] \quad (1)$$

i is an integer type which plays no part in bond pricing except to uniquely identify the bond in an inventory which we describe below; C is the coupon payment amount; n is payment frequency per annum; T is the time to maturity in years; and M is the face value due at maturity. The sum of the net present value of these cash flows, C and M , is the price of the bond. Thus, the fair value, $P(b_i, r)$, of a bond, b_i , is functionally defined follows:

$$P(b_i, r) = \sum_{t=1}^{n \times T} \frac{C}{(1 + r_t)^{t/n}} + \frac{M}{(1 + r_T)^T} \quad (2)$$

The parameter, r , is the time-dependent yield or yield curve, the general discussion of which is beyond the scope of this paper. For simplicity sake without loss of generality, we use the United States Treasury on-the-run bond yield curve and interpolate between the tenors (i.e., bond maturity dates) using polynomial curve fitting.

It is worth noting that Equation 2 informs a number of different quantitative finance calculations which is one of the reasons we have chosen to model bonds. The first

term of the right hand side of Equation 2 is an annuity embedded in the bond's price. The bond price also is used to calculate the conversion factor for bond futures and the fair value of bond forwards and bond options. Swap contracts, the most heavily traded financial derivative in the world [14], involve discounted cash flows similar to Equation 2. Indeed, the bond price itself may also be used in swap valuation. The point is that Equation 2 underlies and plays a part in a number of other quantitative finance problems and for this reason bond pricing is important as a kind benchmark computation.

A portfolio is a collection instruments, in our case, bonds. The fair value, $P(\phi)$, of a portfolio, ϕ , with a basket of Q bonds is functionally defined as follows:

$$P(\phi) = \sum_{q=1}^Q P(b_{\phi(j,q)}, r) \quad (3)$$

In other words, the portfolio's value is the sum over the value of all its constituent bonds.

3.2 Bond generation algorithm

We generate simple bonds that model a wide range of computational scenarios. The goals are to 1) produce a sufficient number of bonds to mimic realistic fixed income portfolios and 2) avoid biases in commercial-grade bonds that depend on prevailing market conditions. Specifically, we have the collections, $\vec{n} = \{1, 4, 12, 52, 365\}$, $\vec{T} = \{1, 2, 3, 4, 5, 7, 10, 30\}$, and $\vec{\delta} = \{0.005, 0.01, 0.02, 0.03, 0.04, 0.05\}$. We derive the parameters for a bond object from the bond generator equations below:

$$M = 1000 \quad (4a)$$

$$n = \vec{n}[\bullet] \quad (4b)$$

$$T = \vec{T}[\bullet] \quad (4c)$$

$$C = M / T \times \delta[\bullet] \quad (4d)$$

where $\bullet$ is an integer uniform random deviate in the range of $[0, s)$; and s is the size of the respective collection. We invoke Equations 4a - 4d a total of 5,000 times to produce the bond inventory, V , which we store in an indexed document-oriented database that we describe later.

We generate a portfolio by first selecting its size, that this, the number of bonds, Q , as per the equation below.

$$Q = v + \sigma \times \eta \quad (5)$$

η is a Gaussian deviate with mean of zero and one standard deviation. σ and v are configurable parameters set to 60 and 20, respectively. Finally, we construct a

basket of size, Q , bonds for a portfolio, ϕ_j , per the equation below.

$$i = \bullet \quad (6)$$

where $\bullet$ is an integer uniform random deviate in the range of $[1,|V|]$ and $|V|$ is the size of the bond inventory. The universe, U , of bond portfolios are also stored in a persistent repository.

3.3 IO design

We store the inputs, the bond and portfolio objects, b_i and ϕ_j in MongoDB, a document-oriented database. [20] We also store the outputs, the portfolio prices, in the database. A portfolio document, ϕ_j , does not contain the bond, b_i , directly but a reference to the bond, namely, the bond's primary key, i . In other words, the database is in third object normal form [17,19]. This means that to evaluate Equation 3 end-to-end, $2+Q$ accesses are required: one to fetch ϕ_j , Q to retrieve each b_i , and finally one to update the portfolio with the price.

3.4 Pricing algorithms

All the pricing algorithms, serial and parallel, work with the database in the same way. They retrieve from the database the same subset, $U' \subseteq U$, of random portfolios and then retrieve the bonds that have been randomly assigned to a portfolio. The algorithms evaluate Equation 2 and Equation 3, and then update the database with the result of Equation 3.

3.5 Serial algorithms

There are two serial algorithms. Both algorithms price portfolios serially using a triple-nested `foldLeft` function, the inner `foldLeft` for Equation 2, the middle `foldLeft` for Equation 3, and the outer `foldLeft` passes over all portfolios in U' . One serial algorithm fetches a bond from the database only when the bond is needed by the pricing function literal. The other serial algorithm, the "memory-bound" algorithm, works the same way except it pre-fetches into memory all the bonds that are needed by the portfolios.

3.6 Parallel naïve algorithm

The naïve parallel algorithm is so-called because it "naively" assigns one portfolio to a mapping function whose inner `foldLeft` evaluates Equation 2 and whose outer `foldLeft` evaluates Equation 3. The H-S function, `mapreduceBasic` method, relates portfolios in U' to two functions: a mapping function and a reducing function. The prototype for our mapping function is below.

```
def mapping(portfId: Int,
            r: List[Double]):
    List[(Int, Result)]
```

The parameter, *portfId*, is the portfolio id and the parameter, *r*, is the time-dependent yield curve. The return result is a list with one element, a two-tuple that has the portfolio id and the pricing result. The prototype for the reducing function is below

```
def reducing(portfId: Int,
            vals: List[Result]):
    List[Result]
```

Note that this function is trivial: it just needs to return the only element in the parameter, *vals*, list.

3.7 Parallel memory-bound algorithm

The parallel memory-bound algorithm is nearly identical to the naïve algorithm. The difference is that the bonds are first pre-loaded into memory from the database, just like the serial memory-bound algorithm except the transfer of bonds to memory is done in parallel with actors and task-level parallelism. However, the parallel memory-bound algorithm uses task-level parallelism using actors to load the bonds in parallel. The parallel memory-bound reducing function which H-S `mapreduceBasic` requires is the same as the naïve reducing function. The parallel memory-bound mapping function prototype is below.

```
def mapping(portfId: Int,
            bonds: List[SimpleBond]):
    List[(Int, Result)]
```

3.8 Parallel coarse-grain algorithm

The parallel coarse-grain algorithm uses the same mapping and reducing functions as the naïve algorithm. The difference is the coarse-grain algorithm uses the H-S `coarseMapReduce` method which takes two extra parameters: the maximum number of mappers and the maximum number of reducers. We set the number of mappers and the number of reducers both to the number of processors which we get from the Java class, *Runtime*. The `getRuntime().availableProcessors()` method of this class returns the number of hyper-threads, not the number of cores or execution units. [4] As far as we know, it is not possible to get the number of actual hardware execution units except from the OEM data sheets.

3.9 Parallel fine-grain algorithm

The parallel fine-grain algorithm is similar to the naïve algorithm. The difference in this case is its map function uses a single (non-nested) `foldLeft` to evaluate

Equation 2. The reduce function evaluates Equation 3. Thus, the prototype for the map function we pass to the H-S *mapreduceBasic* method is below.

```
def mapping(portfId: Int,
           bondId: Int):
    List[(Int, Result)]
```

The *bondId* parameter is the bond id which the mapping function uses to retrieve the bond object. However, since bond id cannot be known in advance, it corresponding portfolio was pre-loaded prior to invoking the mapping function. This pre-load time is taken into account in the runtime measurement. The portfId parameter is only used to link the bond price result with its portfolio which allows the reducer to reduce the bond prices properly.

The reducing function sums all the bond prices by their portfolio id. The reducing function prototype is the same as the naïve algorithm.

4 Experimental design

In this section we describe the experimental design, including the hardware environment, trials, and speedup calculations.

4.1 Environment

The environment consisted of three hardware platforms of different Intel multicore processors. The table below gives the system configurations and year of introduction in each case.

Table 1. Platforms

CPU	Clock	Cores	Threads	RAM	Year
W3540	2.93	4	8	4 GB	2009
i5-650	3.20	2	4	4 GB	2010
T3300	2.50	2	2	3 GB	2008

The clock is in giga-hertz and the year is the year the processor was introduced by Intel. The W3540 and i5 systems run Microsoft Windows 7 Professional, Service Pack 1. The T3800 runs Microsoft Windows XP, Service Pack 3. The code is compiled by Eclipse 3.7 [5] using the Scala IDE plugin [28] version 2.0.0 with the 64-bit JVM on the W3540 and i5 and the 32-bit JVM on the T8300.

4.2 Trials

To obtain statistically conservative results [8], we use the median result of 11 to 22 trials in which we observe the runtime performance for an ensemble of workloads of U' portfolios randomly selected portfolios from a total of $U=100,000$ portfolios. Any given workload in the ensemble contains $u=2^x$ portfolios where $x=\{0,1,2,3,...10\}$ for each trial. Since the mean number of bonds in each portfolio is 60 with a standard

deviation of 20. The largest workload analyzes $u=1024$ portfolios with an average of about 60,000 bonds total.

We measured the serial algorithm time, T_1 , for processing an ensemble using each of the platforms listed in Table 1 above. We also measured the map-reduce parallel algorithm time, T_N , for naïve, coarse-grain, and fine-grain map-reduce algorithms. In the case of the memory-bound algorithms, we have given the compute and the I/O runtimes separately. The memory-bound compute measurement is the median runtime to value the portfolios. The memory-bound I/O measurement is the median runtime to load the bonds into memory.

To get a better understanding of how IO might be impacting the process, we furthermore separated out compute and IO phases by first loading all the bonds into memory before processing them in both serial and map-reduce cases.

4.3 Speed-up calculations

Given T_1 and T_N , we have the speed-up, R .

$$R = T_1 / T_N \quad (7)$$

N is going to be the number of cores or execution units, not the number of hyper-threads.

With R we have, e , the efficiency.

$$e = R / N \quad (8)$$

5 Results

We run trials from $u=1..1,024$ bond portfolios.

5.1 Parallel naïve results

The results for the naïve (*), memory-bound compute (+), and memory-bound I/O-only ($\triangle$) measurements for the W3540 are summarized in the graph below.

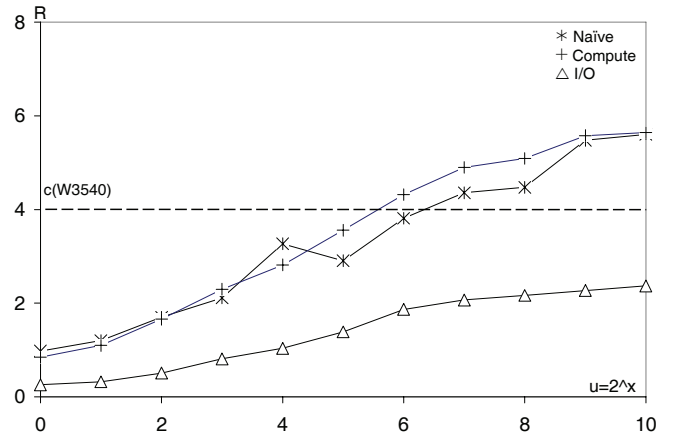


Figure 1. W3540 results

Note that speedup, R , above the dashed line is superlinear and super-efficient performance. The corresponding numerical data for the W3540 for

$u=1,024$ bond portfolios for the naive algorithm is in the table below.

Table 1. W3540 with $u=1,024$ bond portfolios

	T_N	R	e
Naive	8.58	5.64	141%
Memory-bound compute	7.13	5.60	140%
Memory-bound I/O	2.50	2.37	59%

The naive algorithm results for the i5 are summarized in the graph below.

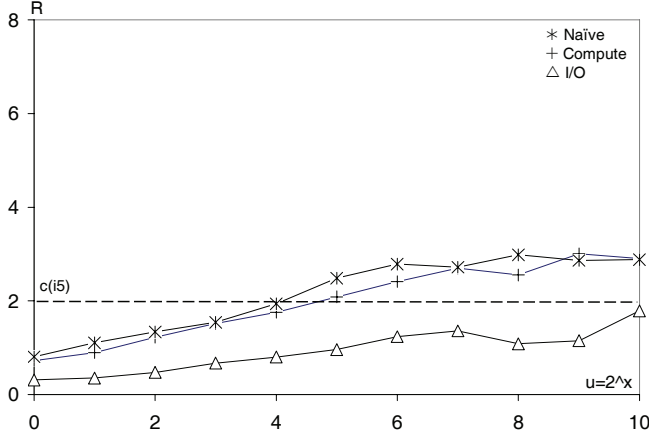


Figure 2. i5 results

The corresponding numerical data for the i5 for $u=1,024$ bond portfolios for the naive algorithm is in the table below.

Table 2. i5 with $u=1,024$ bond portfolios

	T_N	R	e
Naive	16.87	2.90	145%
Memory-bound compute	13.42	2.88	144%
Memory-bound I/O	3.54	1.79	89%

The naive algorithm results for the T8300 are summarized in the graph below.

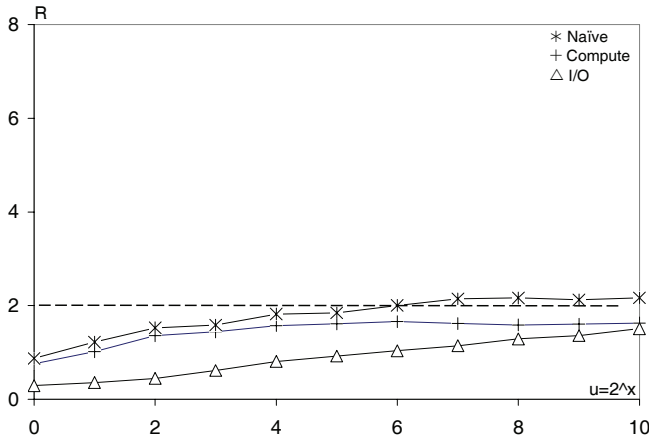


Figure 3. T8300 results

The corresponding numerical data for the T8300 for $u=1,024$ bond portfolios for the naive algorithm is in the table below.

Table 3. T8300 with $u=1,024$ bond portfolios

	T_N	R	e
Naive	73.77	1.63	81%
Memory-bound compute	67.34	2.16	108%
Memory-bound I/O	8.07	1.51	75%

5.2 Parallel fine-grain results

The composite fine-grain results for the W3540 (x), i5-650 (□), and 8300 (◇) are summarized in the graph below.

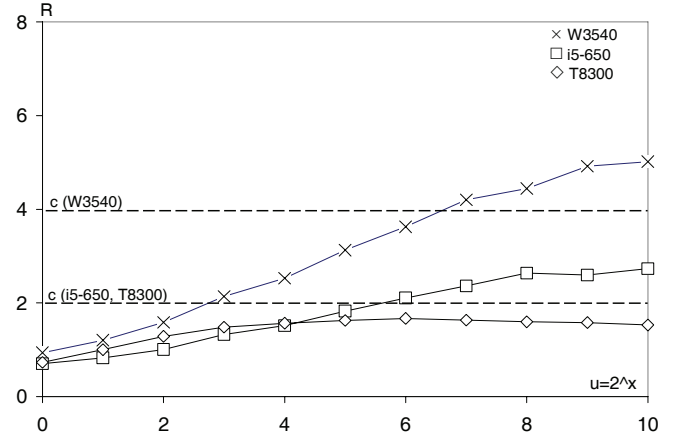


Figure 4. Fine-grain results

The corresponding numerical data for the W3540, i5-650, and T8300 for 1,024 bond portfolios for the naive algorithm is in the table below.

Table 4. Fine-grain with $u=1,024$ bond portfolios

	T_N	R	e
W3540	9.65	5.02	125%
i5-650	17.93	2.73	137%
T8300	78.52	1.53	76%

5.3 Parallel coarse-grain results

The coarse-grain results for the W3540, i5, and T9300 are summarized in the graph below.

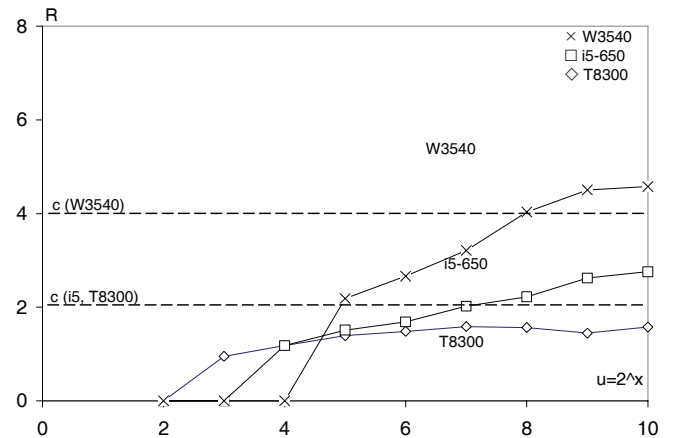


Figure 5. Coarse-grain results

The corresponding numerical data for the W3540, i5-650, and T9300 for 1,024 bond portfolios for the naive algorithm is in the table below.

Table 5.

	T_N	R	e
W3540	9.16	5.29	132%
i5-650	19.55	2.50	125%
T8300	79.20	1.51	76%

6 Discussion

The naïve algorithm appears to be the best performing overall end-to-end, achieving super-linearity and super-efficiency for levels of u , depending on the processor type. For instance, the more modern processors, the W3540 and i5, realize super-linearity and super-efficiency for u as small as 64.

Most interestingly, I/O is broadly sub-linear which, by itself, is not surprising. However, I/O does not appear to be a processing bottleneck since the difference between compute and memory-bound compute plus memory-bound I/O over the range of u appears to be insignificant.

7 Conclusion

This investigation of map-reduce for computational finance lead us to consider other avenues for future work. First, we would like to explore changes to H-S to support multiprocessor parallelism. Second, there are open questions on how to “shard” or parallelize the data. Finally, we had briefly mentioned Scala’s parallel collections. A very worthwhile study might examine parallel collections and compare the coding style and performance to H-S map-reduce.

8 Acknowledgements

This research has been funded in part by grants from the National Science Foundation, Academic Research Infrastructure award number 0963365 and Major Research Instrumentation award number 1125520.

9 References

- [1] <http://code.google.com/p/scaly/> retrieved on 30 January 2012
- [2] Crosman, P., “Bloomberg Uses GPs to Speed Up Bond Pricing,” *Wall Street Technology*, 24 September 2009
- [3] Dean, J. and Ghemawat, S. “MapReduce: Simplified Data Processing on Large Clusters,” OSDI 2004
- [4] [http://docs.oracle.com/javase/6/docs/api/java/lang/Runtime.html#availableProcessors\(\)](http://docs.oracle.com/javase/6/docs/api/java/lang/Runtime.html#availableProcessors()) retrieved on 29 January 2012
- [5] <http://eclipse.org/> retrieved on 29 January 2012
- [6] Fabozzi, F.J., Mann, S.V., *Introduction to Fixed Income Analytics*, 2nd ed. Wiley, 2010
- [7] <http://foxweb.marist.edu/users/ron.coleman/> retrieved on 30 January 2012
- [8] Georges, A., et al, “Statistically Rigorous Java Performance Evaluation,” *OOPSLA '07*, October 21-25, Montreal, Quebec, Canada
- [9] Gustafson, J.L., “Fixed Time, Tiered Memory, and Superlinear Speedup,” *Proceedings of the Fifth Distributed Memory Computing Conference*, 8-12 Apr 1990, p.1255-1260
- [10] <http://hadoop.apache.org/> retrieved on 29 January 2012
- [11] Hill, M.D., Marty, M.R., “Amdahl’s Law in the Multicore Era,” IEEE Computer Society, July 2008
- [12] Haller, P., Sommers, F., *Actors in Scala*, Artime, 2011
- [13] Hewitt, C.; Bishop, P., and Steiger, R., “A Universal Modular Actor Formalism for Artificial Intelligence”, *IJCAI*, 1973
- [14] Hull, J., *Options, Futures, and Other Derivatives*, Prentice Hall, 1989
- [15] “Hyper-Threading Technology,” *Intel Technology Journal*, Vol. 6, Issue 01, 14 February 2002
- [16] Lammal, R., “Google’s MapReduce Programming Model Revisited,” <http://www.cs.vu.nl/~ralf/MapReduce/paper.pdf> retrieved on 11 Mar 2012
- [17] Lodhi, F., Mehdi, H., “Normalization of object-oriented design,” *Seventh International Multi-Topic Conference*, Lahore Pakistan, 8-9 Dec 2003, p. 446-450
- [18] Ma, Z., Gu, L., “The Limitation of MapReduce: A Probing Case and a Lightweight Solution,” *Proc. of the 1st Intl. Conf. on Cloud Computing*, Nov. 21-26, 2010, Lisbon, Portugal
- [19] Merunka, V., et al, “Normalization Rules of the Object-Oriented Data Model,” *EOMAS '09* Proceedings of the International Workshop on Enterprises & Organizational Modeling and Simulation
- [20] <http://www.mongodb.org/> retrieved on 30 January 2012
- [21] NVIDIA, “Computational Finance on the GPU,” *GPU Technology Conference*, San Jose, CA, 30 Septemter – 2 October 2009

- [22] NVIDIA, "JP Morgan Speeds Risk Calculations with NVIDIA GPUs", *HPC Wire*, 4 August 2011
- [23] Odersky, M., et al, *Programming in Scala*, Artima, 2008
- [24] Owens, J.D., et al, "GPU Computing," *Proceedings of the IEEE*, Vol. 96, No. 5, May 2008
- [25] Prokopec, A. et al, "A Generic Parallel Collection Framework," EPFL, *InfoScience* 2011, 2010-7-31
- [26] Schmerken, I., "Wall Street Accelerates Options Analysis with GPU Technology," *Wall Street Technology*, 11 Mar 2009
- [27] Ranger, C., et al, "Evaluating MapReduce for Multi-core and Multiprocessor Systems," *HPCA*, 2007
- [28] <http://scala-ide.org/>, retrieved on 29 January 2012
- [29] Tuckman, B., *Fixed Income Securities*, 2nd ed., Wiley Finance, 2002
- [30] Tsang, E.P.K., Martinez-Jaramillio, "Computational Finance," *IEEE Computational Intelligence Society*, August 2004
- [31] Woo, J., Xu, Y., "Market Basket Analysis Algorithm with Map/Reduce of Cloud Computing," *2011 International Conference on Parallel and Distributed Processing Techniques and Applications*, WORLDCOMP'11 July 18-21, 2011, Las Vegas Nevada, USA
- [32] Zhang, Y., et al, "Parallel option pricing with BSDEs method on MapReduce," *Third International Conference on Computer Research and Development (ICCRD)*, Shanghai, 11-13 Mar 2011, p289-293